



Actividad 7

Grafos, DFS, BFS y pandas

Entrega

- **Lugar:** Repositorio personal de GitHub — Carpeta: Actividades/AC7
- **Fecha máxima de entrega:** 11 de Junio 17:20
- **Ejecución de actividad:** La Actividad será ejecutada **únicamente** desde la terminal del computador. Los *paths* relativos utilizados en la Actividad deben ser coherentes con esta instrucción, y no pueden modificarse.

Importante: Antes de comenzar, comprueba que Git este funcionando correctamente en tu repositorio privado. Para esto, **sube los archivos base de la actividad de inmediato** (*add*, *commit*, *push*). Se espera que en esta actividad (así como en las demás actividades y tareas) utilices Git a lo largo de **todo tu desarrollo** como una herramienta, no sólo como un método de entrega. Es por esto que recomendamos enfáticamente que vayas subiendo tus cambios constantemente (*push*), ya que **problemas de último minuto** relacionados con la entrega y Git **no serán considerados**.

Introducción

DFS y BFS son ambos algoritmos de búsqueda útiles en muchos escenarios. En la actividad de hoy los aplicaremos para buscar un objetivo en un laberinto y compararemos su eficacia usando pandas.

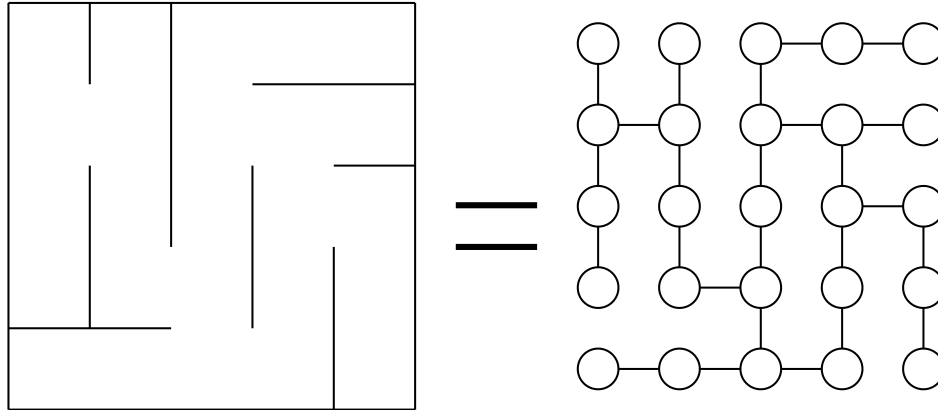
Archivos

El código base tiene 3 archivos. Debes subir los archivos con el tag **Entregar** al repositorio para que sean evaluados. Los cambios en los otros archivos no serán considerados y no es necesario que los subas al repositorio.

- **No modificar** `laberinto.py`: Contiene el código que genera un laberinto aleatorio, y la implementación de la clase `Celda`, que funciona como un `Nodo` de un `Grafo`.
- **Modificar** **Entregar** `busqueda.py`: Contiene la implementación básica de DFS (iterativo) y BFS, además del método `reconstruir_ruta`. Se modificará este archivo para la parte 1 de la actividad.
- **Modificar** **Entregar** `estadisticas.py`: Contiene el código que mide las estadísticas de usar DFS y BFS usando pandas. Tendrás que completar los métodos de este archivo para la parte 2 de la actividad.

Laberinto modelado como grafo

El problema de la actividad de hoy tiene un laberinto con forma de grilla rectangular. Este laberinto puede ser modelado usando tanto una grilla como un grafo, donde cada celda de la grilla es un nodo del grafo.



En el archivo `laberinto.py` encontrarás las clases `Celda` y `Laberinto`. Estas son usadas para generar un laberinto aleatorio y proveen métodos que necesitarás y otros que te pueden ser útiles.

Muchos de los métodos y atributos de este archivo son utilizados para generar el laberinto aleatorio, por lo que no necesitas saber como funcionan. Si tienes curiosidad de cómo funciona la generación de laberinto aleatorio, busca el algoritmo de [Kruskal](#). A continuación se describen solo los métodos que te serán de utilidad.

- **No modificar** `class Celda`: Corresponde a un nodo del grafo del laberinto. Contiene algunos métodos relacionados a la generación del laberinto aleatorio. Tú solo debes preocuparte de los métodos listados a continuación:
 - `def __init__(self, fila: int, columna: int)`: Inicializa una celda del laberinto con los valores `fila` y `columna` recibidos como parámetros. Además, acá inicializa la lista de vecinos `self.conexiones`.
 - `def __repr__(self) -> str`: retorna `"(fila, columna)"` para que la celda se imprima de forma más legible en los prints.
 - `def conexiones_ordenadas(self, fin: "Celda") -> list["Celda"]`: Retorna una copia de la lista de conexiones de la celda, ordenada según su distancia a la celda `fin`. Ordena de mayor a menor distancia, y no modifica la lista original.
- **No modificar** `class Laberinto`: Clase que modela el laberinto y contiene las celdas. Se genera de forma aleatoria al momento de inicializar. El laberinto generado puede tener más de una ruta posible entre 2 celdas.
 - `def __init__(self, n_filas: int, n_columnas: int, redundancia: float)`: Inicializa un laberinto aleatorio con el número de filas y columnas dado.

El input `redundancia` es un valor entre 0 y 1, y se utiliza para definir que tan conectado es el laberinto. Cuando vale 0, el laberinto tiene un sola ruta posible entre cualquier par de celdas, y al aumentar el valor de la redundancia, se generan más rutas que conectan las celdas

(equivalente a botar muros en el laberinto). Cuando vale 1, se botan todos los muros y el laberinto es solo un rectángulo.

- `def imprimir_laberinto(self):` Imprime el laberinto en consola.
- `def obtener_celda(self, fila: int, columna: int) -> Celda:` Retorna la celda correspondiente a la posición indicada usando fila y columna.

Parte 1. Búsqueda

En esta sección debes completar las funciones del archivo `busqueda.py`. Tu objetivo es hacer que ambos métodos de búsqueda (DFS y BFS) retornen la ruta entre las celdas indicadas junto con el `set` de celdas vistas por el algoritmo correspondiente. De esta manera en la sección 2 podremos comparar el largo de las rutas y el número de celdas vistas por cada algoritmo.

La forma en la que se construye la ruta entre 2 puntos usando DFS o BSF es recordar la celda anterior a cada celda en la búsqueda, o la “celda padre”. Una vez terminada la exploración del grafo, es posible reconstruir la ruta desde la celda final hasta la celda inicial usando las celdas padre. Una vez hecho esto, basta con revertir el camino.

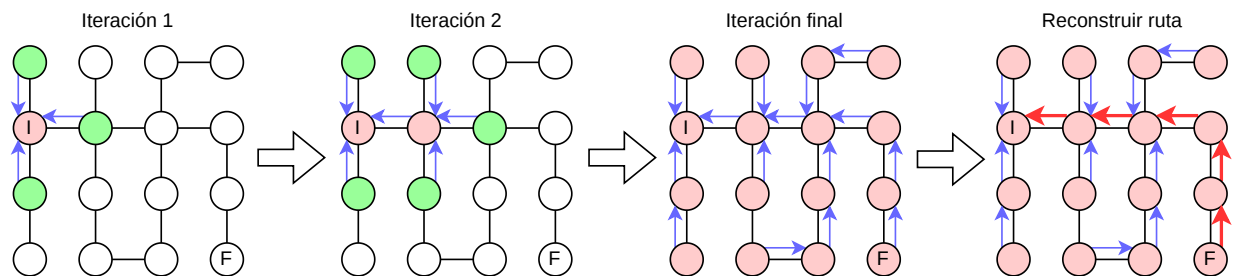


Figura 1: En cada iteración las celdas agregados al stack de DFS recuerdan la celda que las generó

A continuación se detalla lo que hace cada método y lo que necesitas completar.

- **Modificar** `def reconstruir_ruta(celda_padre: dict, fin: Celda) -> list:`
Método que reconstruye la ruta entre la celda final y inicial usando las celdas padre. Recibe como input:
 - `celda_padre`: Diccionario donde las llaves y valores son celdas del laberinto.
`celda_padre[celda1] = celda2` indica que `celda2` descubrió a `celda1` durante la búsqueda (`celda2` es la celda padre de `celda1`)
 - `fin`: indica la celda a la cual se quiere llegar en la búsqueda.

Retorna la ruta entre la celda inicial de la búsqueda y la final. Es una lista donde cada elemento es una `Celda`, el primer elemento es la celda inicial, y el último es la celda final.

- **Modificar** `def buscar_dfs(inicio: Celda, fin: Celda, ordenar_conexiones: bool) -> tuple[list, set]:`
Función que utiliza DFS iterativo para encontrar el camino entre las celdas `inicio` y `fin`. Recibe como input:
 - `inicio`: Celda desde la cual parte la búsqueda.
 - `fin`: Celda a la cual se busca llegar desde inicio.

- `ordenar_conexiones`: Booleano que indica si se deben ordenar las conexiones de las celdas al buscar.

Retorna:

- Ruta: lista con celdas que corresponde a la ruta reconstruida con el método `reconstruir_ruta`
- Visitados: set con celdas visitadas por el algoritmo DFS

En esta función debes:

- 1) **Guardar las celdas padre**: Modifica la función para que guarde la celda padre de cada celda descubierta en un diccionario que pueda utilizar `reconstruir_ruta`.
- 2) **Terminar de iterar anticipadamente**: Ya que solo estamos buscando la celda final, no queremos recorrer todo el grafo. Una vez que encuentres la celda final, debes dejar de iterar, luego debes llamar a `reconstruir_ruta` para obtener la ruta, y retornarla junto con los nodos visitados.
- 3) **Ordenar conexiones**: Solo cuando cuando el input `ordenar_conexiones` sea `True` debes cambiar ligeramente el comportamiento de DFS para que priorice buscar en dirección a la celda final.

Para hacer esto, utiliza el método `celda.conexiones_ordenadas(fin)` para obtener una lista ordenada de conexiones según su distancia a la celda `fin`. La lista obtenida tiene orden descendente de distancia ya que la última celda en ser insertada al `stack` de DFS será la primera celda explorada.

- **Modificar** `def buscar_bfs(inicio: Celda, fin: Celda) -> tuple[list, set]`: Función que busca la celda final desde la celda inicial usando BFS. A diferencia de `buscar_dfs`, ordenar las conexiones no hará ninguna diferencia en la búsqueda ya que este algoritmo busca celdas desde las más cercanas hasta las más lejanas al inicio, por lo que no recibe el argumento `ordenar_conexiones`.

De forma similar a la función de DFS, modifica `buscar_bfs` para que:

- Guarde las celdas padre de cada celda descubierta.
- Termine de iterar anticipadamente.
- Reconstruya la ruta y la retorne junto con el set de celdas visitadas.

Parte 2. Estadísticas

En esta parte trabajarás con el archivo `estadisticas.py` y utilizarás los algoritmos de la parte 1, y la librería `pandas` para comparar su eficiencia y optimalidad. En particular, nos interesa medir el largo de la ruta generada y el número de celdas visitadas por `buscar_dfs` al cambiar el valor `ordenar_conexiones`. Utilizaremos como referencia los valores obtenidos con `buscar_bfs` ya que sabemos que genera siempre la ruta óptima pero visita muchos nodos.

Se recomienda asegurarse de que los tests de la parte 1 sean exitosos antes de implementar esta sección ejecutando:

```
python -m unittest -v -b tests_publicos.tests_busqueda.VerificarDFS
```

y

```
python -m unittest -v -b tests_publicos.tests_busqueda.VerificarBFS
```

A continuación se explican los contenidos del atchivo `estadisticas.py` y qué debes hacer:

1. **No modificar** **Experimento**: Es una `namedtuple` que contiene todos los valores necesarios para iniciar un experimento. Contiene los valores:
 - `alto` (`int`): el número de filas del laberinto a usar.
 - `ancho` (`int`): el número de columnas del laberinto a usar.
 - `redundancia` (`float`): el valor de redundancia usado para crear el laberinto.
 - `repeticiones` (`int`): la cantidad de veces que se debe repetir el experimento en el mismo laberinto pero con posiciones aleatorias distintas.
2. **Modificar** **def correr_experimento**(experimento: Experimento) -> pd.DataFrame: Función que recibe como input un objeto de tipo `Experimento` y retorna un `DataFrame` de `pandas` con los resultados. Debes completar este método para que haga lo siguiente:
 - Inicializar un `Laberinto` con las dimensiones y redundancia indicadas en `experimento`.
 - Una vez por cada repetición indicada debe:
 - Elegir celdas inicial y final en el laberinto aleatoriamente usando el método `laberinto.generar_celda_aleatoria()`
 - Ejecutar `buscar_dfs` con y sin ordenar conexiones, y ejecutar `buscar_bfs` una vez.
 - Registrar el largo de cada ruta y el número de nodos visitados en cada búsqueda.
 - Retornar un `DataFrame` de `pandas` donde cada fila es una repetición del experimento y que tenga las columnas:
 - `"redundancia"`: Valor de la redundancia usado en el experimento.
 - `"visitados_ordenado"`: Número de nodos visitados por `buscar_dfs` ordenando las conexiones.
 - `"visitados_sin_ordenar"`: Número de nodos visitados por `buscar_dfs` sin ordenar las conexiones.
 - `"visitados_bfs"`: Número de nodos visitados por `buscar_bfs`.
 - `"largo_ordenado"`: Largo de la ruta generada por `buscar_dfs` ordenando las conexiones.
 - `"largo_sin_ordenar"`: Largo de la ruta generada por `buscar_dfs` sin ordenar las conexiones.
 - `"largo_bfs"`: Largo de la ruta generada por `buscar_bfs`.
3. **Modificar** **def normalizar_datos**(datos: pd.DataFrame) -> None: Esta función toma como input un `DataFrame` con el formato de output de la función anterior y le agrega columnas normalizadas *in place* (Modifica el dataframe original, no copia los datos).

Debes agregar las columnas `"norm_visitados_ordenado"`, `"norm_visitados_sin_ordenar"`, `"norm_largo_ordenado"` y `"norm_largo_sin_ordenar"` al dataframe.

Cada valor normalizado se obtiene dividiendo el valor sin normalizar por el valor obtenido por `buscar_bfs`. Por ejemplo si en una fila del dataframe tenemos `visitados_ordenado = 45` y `visitados_bfs = 100`, el valor `norm_visitados_ordenado` debería ser 0.45, lo que significa que DFS ordenando conexiones visitó solo un 45 % de las celdas que BFS visitó.

Una vez completada la parte 2, puedes ejecutar `python3 estadisticas.py` para ver las estadísticas del programa y comprobar que la optimización hace una diferencia importante en la búsqueda.

Desafío opcional

Este desafío no es evaluado

Como desafío opcional, puedes usar `matplotlib` para generar un gráfico que muestre los datos de las estadísticas como te parezca apropiado. Recomiendo investigar los distintos tipos de gráficos que maneja la librería.

Notas

- No puedes hacer *import* de otras librerías externas a las entregadas en el archivo.
- Cada archivo tiene una sección `main` donde puedes probar las funciones que necesites.
- Tu actividad será evaluada usando tus archivos modificados `busqueda.py` y `estadisticas.py`
- Recuerda que la ubicación de tu entrega es en **tu repositorio de Git**. En la rama (*branch*) por defecto del repositorio: `main`.
- Se recomienda completar la actividad en orden.
- Recuerda que esta evaluación presenta corrección **automatizada**. Si entregas un código que se cae al momento de correr los *tests*, será evaluado con 0 puntos.
- Siéntete libre de agregar nuevos `print` en cualquier lugar para revisar objetos, modificaciones, etc... Es una herramienta muy útil para encontrar errores. Y si aparece un error inesperado, ¡léelo! Intenta interpretarlo.

Objetivo de la actividad

- Aplicar los algoritmos de DFS y BFS en un problema de búsqueda.
- Utilizar `pandas` para procesar datos de un experimento.
- Probar código mediante la ejecución de *test*.
- Optimizar algoritmos de búsqueda basado en el problema.

Ejecución de *tests*

En esta actividad se provee de varios archivos `.py` los cuáles contiene diferentes *tests* que ayudan a validar el desarrollo de la actividad. Para ejecutar estos *tests*, **primero debes posicionar tu terminal/consola en la carpeta de la actividad (Actividades/AC7)**. Luego, desde esta misma, debes escribir el siguiente comando para ejecutar todos los *tests* de la actividad:

```
python3 -m unittest discover tests_publicos -v -b
```

Existe una clase de test por cada parte del enunciado. Si quieres testear ejecutar una parte en particular, puedes hacer usando el comando:

```
python3 -m unittest -v -b <carpeta>.<archivo>.<NombreClase>
```

Por ejemplo, si quieres testear la parte 1:

```
python -m unittest -v -b tests_publicos.tests_busqueda.VerificarDFS
```

Importante: recuerda que si `python3` no funciona, probar con el comando específico de tu computador. Este puede ser `py`, `python`, `py3` o `python3.12`.